

CSA0619-DESIGN ANALYSIS OF ALGORITHM

CO5 AT1 ANSWERS

BY ANTONY MAGRIN J(192572103)

Q3. Password Generation — Combinatorial Backtracking Problem

1. Problem Definition and Constraints

Input requirements:

- N — desired password length
- An alphabet Σ , partitioned into categories: uppercase A–Z, lowercase a–z, digits 0–9, specials !@#\$%^&*()-_+=
- A set of required categories the password must contain at least one character from (here: uppercase, digit — extendable to lowercase/special)

Constraints:

- **C1 (Length):** $\text{len}(\text{password}) == N$
- **C2 (Uppercase):** at least one character from A–Z
- **C3 (Digit):** at least one character from 0–9
- **C4 (No repetition):** all N characters are pairwise distinct $\rightarrow N \leq |\Sigma|$

2. Effect on the Search Space

Without constraints, choosing N characters with repetition from $|\Sigma|$ symbols gives $|\Sigma|^N$ strings. Banning repeats (C4) shrinks this to the permutation count $P(|\Sigma|, N) = |\Sigma|! / (|\Sigma| - N)!$ — e.g. for $|\Sigma|=76$, $N=16$ that's $\approx 2.27 \times 10^{29}$ (verified by the program's "DEMO 3"). Requiring specific categories (C2, C3) further restricts this to a subset of those permutations. The key algorithmic insight: rather than generating the full permutation space and filtering, backtracking should **detect infeasibility early** and cut branches before they're built.

3. Backtracking Design

State per node: path (chars chosen), used (set, for C4), satisfied (which of C2/C3 are already met).

Pruning rules applied before recursing:

- **P1 – No-repeat pruning:** skip any candidate already in used (enforces C4 without ever exploring duplicate branches).
- **P2 – Feasibility pruning:** if (required categories still missing) > (slots remaining after this choice), the branch can never reach a valid leaf — cut it.
- **P3 – Alphabet-exhaustion pruning:** if fewer unused characters remain than empty slots left, the branch is dead — cut it.

Goal test (only at a full-length leaf): all required categories present. Length and no-repeat are guaranteed by construction, not re-checked.

4. Completeness & Correctness Justification

- **Completeness:** every character not excluded by P1–P3 is tried, and backtracking (undo + continue) systematically explores the entire remaining tree, so no valid password is skipped.
- **Correctness:** P1 makes duplicates unreachable (C4 always holds); the goal test enforces C2/C3; length is fixed by recursion depth (C1).
- I verified this against a brute-force `itertools.permutations` count on a small alphabet: **both methods found exactly 312 valid passwords** out of 360 total permutations — confirming the search is both complete and sound.

5. Execution Results

DEMO 1 (small alphabet 'AB12ab', N=4): backtracking = 312, brute-force
= 312 → match ✓
DEMO 2 (full 76-char alphabet, randomized order for variety):
N=8 -> 'UxhWl3fN' (len ✓, unique ✓, upper ✓, digit ✓)
N=12 -> 'gti_1sqom-WH'
N=16 -> '!*(b%DjWhU9ZFvp7'

Complexity: worst case (no valid categories found early) is bounded by $P(|\Sigma|, N)$, but pruning makes the practical generation cost near $O(N)$ — one candidate check per position — since P2/P3 discard invalid branches immediately rather than after full construction.